

QUALITY
MANAGEMENT

KEY

CONCEPTS

cost of quality

defect amplification

defect costs

ISO 9001: 2000

reliability

reviews (FTRs)

quality

quality control

sampling

six sigma

software safety

SQA Plan

statistical SQA

The software engineering approach described in this book works toward a single goal: to produce high-quality software. Yet many readers will be challenged by the question: What is software quality?

Philip Crosby [CRO79], in his landmark book on quality, provides a wry answer to this question:

The problem of quality management is not what people don't know about it. The problem is what they think they do know . . .

In this regard, quality has much in common with sex. Everybody is for it. (Under certain conditions, of course.) Everyone feels they understand it. (Even though they wouldn't want to explain it.) Everyone thinks execution is only a matter of following natural inclinations. (After all, we do get along somehow.) And, of course, most people feel that problems in these areas are caused by other people. (If only they would take the time to do things right.)

Some software developers continue to believe that software quality is something you begin to worry about after code has been generated. Nothing could be further from the truth! *Quality management* (often called *software quality assurance*) is an umbrella activity (Chapter 2) that is applied throughout the software process.

Quality management encompasses (1) a software quality assurance (SQA) process; (2) specific quality assurance and quality control tasks (including formal technical reviews and a multitiered testing strategy); (3) effective software engineering practice (methods and tools); (4) control of all software work products

QUICK
LOOK

What is it? It's not enough to talk the talk by saying that software quality is important. You have to (1) explicitly define what is meant when you say "software quality," (2) create a set of activities that will help ensure that every software engineering work product exhibits high quality, (3) perform quality control and assurance activities on every software project, (4) use metrics to develop strategies for improving your software process and, as a consequence, the quality of the end product.

Who does it? Everyone involved in the software engineering process is responsible for quality.

Why is it important? You can do it right, or you can do it over again. If a software team stresses quality in all software engineering activities, it reduces the amount of rework that it must do. That results in lower costs, and more importantly, improved time-to-market.

What are the steps? Before software quality assurance activities can be initiated, it is important to define "software quality" at a number of different levels of abstraction. Once you understand what quality is, a software team must identify a set of SQA activities that will filter errors out of work products before they are passed on.

What is the work product? A *Software Quality Assurance Plan* is created to define a software team's SQA strategy. During analysis, design, and code generation, the primary SQA work product is the formal technical review summary report. During testing, test plans and procedures are produced. Other work products

associated with process improvement may also be generated.

How do I ensure that I've done it right?

Find errors before they become defects! That is, work to improve your defect removal efficiency (Chapter 22), thereby reducing the amount of rework that your software team has to perform.

and the changes made to them (Chapter 27); (5) a procedure to ensure compliance with software development standards (when applicable), and (6) measurement and reporting mechanisms.

In this chapter, we focus on the management issues and the process-specific activities that enable a software organization to ensure that it does the right things at the right time in the right way.

26.1 QUALITY CONCEPTS¹

KEY POINT

Controlling variation is the key to a high-quality product. In the software context, we strive to control the variation in the generic process we apply and the quality emphasis that permeates software engineering work.

Variation control is the heart of quality control. A manufacturer wants to minimize the variation among the products that are produced, even when doing something relatively simple like duplicating DVDs. Surely, this cannot be a problem—duplicating DVDs is a trivial manufacturing operation, and we can guarantee that exact duplicates of the software are always created.

Or can we? We need to ensure the tracks are placed on the DVDs within a specified tolerance so that the overwhelming majority of DVD drives can read the media. The disk duplication machines can, and do, wear and go out of tolerance. So even a “simple” process such as DVD duplication may encounter problems due to variation between samples.

But how does this apply to software work? How might a software development organization need to control variation? From one project to another, we want to minimize the difference between the predicted resources needed to complete a project and the actual resources used, including staff, equipment, and calendar time. In general, we would like to make sure our testing program covers a known percentage of the software, from one release to another. Not only do we want to minimize the number of defects that are released to the field, we'd like to ensure that the variance in the number of bugs is also minimized from one release to another. (Our customers will likely be upset if the third release of a product has 10 times as many

¹ This section, written by Michael Stovsky, has been adapted from “Fundamentals of ISO 9000,” a workbook developed for *Essential Software Engineering*, a video curriculum developed by R. S. Pressman & Associates, Inc. Reprinted with permission.

defects as the previous release.) We would like to minimize the differences in speed and accuracy of our hotline support responses to customer problems. The list goes on and on.

26.1.1 Quality

The *American Heritage Dictionary* defines *quality* as “a characteristic or attribute of something.” As an attribute of an item, quality refers to measurable characteristics—things we can compare to known standards such as length, color, electrical properties, and malleability. However, software, largely an intellectual entity, is more challenging to characterize than physical objects.

Nevertheless, measures of a program’s characteristics do exist. These properties include cyclomatic complexity, cohesion, number of function points, lines of code, and many others discussed in Chapter 15. When we examine an item based on its measurable characteristics, two kinds of quality may be encountered: quality of design and quality of conformance.

Quality of design refers to the characteristics that designers specify for an item. *Quality of conformance* is the degree to which the design specifications are followed during manufacturing.

“People forget how fast you did a job—but they always remember how well you did it.”

Howard Newton

In software development, quality of design encompasses requirements, specifications, and the design of the system. Quality of conformance is an issue focused primarily on implementation. If the implementation follows the design and the resulting system meets its requirements and performance goals, conformance quality is high.

But are quality of design and quality of conformance the only issues that software engineers must consider? Robert Glass [GLA98] argues that a more “intuitive” relationship is in order:

$$\begin{aligned} \text{user satisfaction} &= \text{compliant product} + \text{good quality} \\ &\quad + \text{delivery within budget and schedule} \end{aligned}$$

At the bottom line, Glass contends that quality is important, but if the user isn’t satisfied, nothing else really matters. DeMarco [DEM99] reinforces this view when he states: “A product’s quality is a function of how much it changes the world for the better.” This view of quality contends that if a software product provides substantial benefit to its end-users, they may be willing to tolerate occasional reliability or performance problems.

26.1.2 Quality Control

Variation control may be equated to quality control. But how do we achieve quality control? Quality control involves the series of inspections, reviews, and tests used

throughout the software process to ensure each work product meets the requirements placed upon it. Quality control includes a feedback loop to the process that created the work product. The combination of measurement and feedback allows us to tune the process when the work products created fail to meet their specifications.

A key concept of quality control is that all work products have defined, measurable specifications to which we may compare the output of each process. The feedback loop is essential to minimize the defects produced.

26.1.3 Quality Assurance

Quality assurance consists of a set of auditing and reporting functions that assess the effectiveness and completeness of quality control activities. The goal of quality assurance is to provide management with the data necessary to be informed about product quality, thereby gaining insight and confidence that product quality is meeting its goals. Of course, if the data provided through quality assurance identify problems, it is management's responsibility to address the problems and apply the necessary resources to resolve quality issues.

26.1.4 Cost of Quality

The cost of quality includes all costs incurred in the pursuit of quality or in performing quality-related activities. Cost of quality studies are conducted to provide a baseline for the current cost of quality, identify opportunities for reducing the cost of quality, and provide a normalized basis of comparison. The basis of normalization is almost always dollars. Once we have normalized quality costs on a dollar basis, we have the necessary data to evaluate where the opportunities lie to improve our processes. Furthermore, we can evaluate the effect of changes in dollar-based terms.

Quality costs may be divided into costs associated with prevention, appraisal, and failure. *Prevention costs* include quality planning, formal technical reviews, test equipment, and training. *Appraisal costs* include activities to gain insight into product condition the "first time through" each process. Examples of appraisal costs include in-process and interprocess inspection, equipment calibration and maintenance, and testing.

Failure costs are those that would disappear if no defects appeared before shipping a product to customers. Failure costs may be subdivided into internal failure costs and external failure costs. *Internal failure costs* are incurred when we detect a defect in our product prior to shipment. Internal failure costs include rework, repair, and failure mode analysis. *External failure costs* are associated with defects found after the product has been shipped to the customer. Examples of external failure costs are complaint resolution, product return and replacement, help line support, and warranty work.

As expected, the relative costs to find and repair a defect increase dramatically as we go from prevention to detection to internal failure to external failure costs.

WebRef.

Useful links to SQA resources can be found at www.qualitytree.com/links/index.htm.

? What are the components of the cost of quality?

ADVICE

Don't be afraid to incur significant prevention costs. Rest assured that your investment will provide an excellent return.

FIGURE 26.1

Relative cost of correcting an error

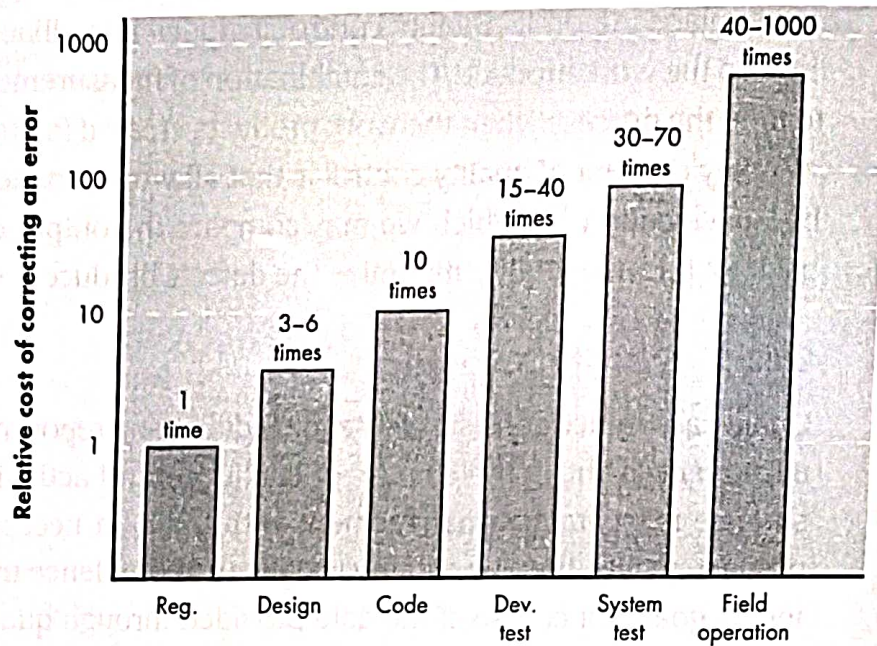


Figure 26.1, based on data collected by Boehm [BOE81] and others, illustrates this phenomenon.

"It takes less time to do a thing right than to explain why you did it wrong."

H. W. Longfellow

26.2 SOFTWARE QUALITY ASSURANCE

Even the most jaded software developers will agree that high-quality software is an important goal. But how do we define quality? A wag once said, "Every program does something right, it just may not be the thing that we want it to do."

Many definitions of software quality have been proposed in the literature. For our purposes, *software quality* is defined as:

? How do we define software quality?

Conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software.

There is little question that this definition could be modified or extended. In fact, the definition of software quality could be debated endlessly. For the purposes of this book, this definition serves to emphasize three important points:

1. Software requirements are the foundation from which quality is measured. Lack of conformance to requirements is lack of quality.
2. Specified standards define a set of development criteria that guide the manner in which software is engineered. If the criteria are not followed, lack of quality will almost surely result.

3. A set of implicit requirements often goes unmentioned (e.g., the desire for ease of use and good maintainability). If software conforms to its explicit requirements but fails to meet implicit requirements, software quality is suspect.

26.2.1 Background Issues

Quality control and assurance are essential activities for any business that produces products to be used by others. Prior to the twentieth century, quality control was the sole responsibility of the craftsperson who built a product. The first formal quality assurance and control function was introduced at Bell Labs in 1916 and spread rapidly throughout the manufacturing world. During the 1940s, more formal approaches to quality control were suggested. These relied on measurement and continuous process improvement [DEM86] as key elements of quality management.

"You made too many wrong mistakes."

Yogi Berra

Today, every company has mechanisms to ensure quality in its products. In fact, explicit statements of a company's concern for quality have become a marketing ploy during the past few decades.

The history of quality assurance in software development parallels the history of quality in hardware manufacturing. During the early days of computing (1950s and 1960s), quality was the sole responsibility of the programmer. Standards for quality assurance for software were introduced in military contract software development during the 1970s and have spread rapidly into software development in the commercial world [IEE94]. Extending the definition presented earlier, software quality assurance is a "planned and systematic pattern of actions" [SCH98] that are required to ensure high quality in software. Many different constituencies have software quality assurance responsibility—software engineers, project managers, customers, salespeople, and the individuals who serve within an SQA group.

The SQA group serves as the customer's in-house representative. That is, the people who perform SQA must look at the software from the customer's point of view. Does the software adequately meet the quality factors noted in Chapter 15? Has software development been conducted according to preestablished standards? Have technical disciplines properly performed their roles as part of the SQA activity? The SQA group attempts to answer these and other questions to ensure that software quality is maintained.

26.2.2 SQA Activities

Software quality assurance is composed of a variety of tasks associated with two different constituencies—the software engineers who do technical work and an SQA group that has responsibility for quality assurance planning, oversight, record keeping, analysis, and reporting.

Software engineers address quality (and perform quality assurance and quality control activities) by applying solid technical methods and measures, conducting formal technical reviews, and performing well-planned software testing. Only reviews are discussed in this chapter. Technology topics are discussed in Parts 1, 2, 3, and 5 of this book.

The charter of the SQA group is to assist the software team in achieving a high-quality end product. The Software Engineering Institute recommends a set of SQA activities that address quality assurance planning, oversight, record keeping, analysis, and reporting. These activities are performed (or facilitated) by an independent SQA group that conducts the following activities:

? What is the role of an SQA group?

Prepares an SQA plan for a project. The plan is developed during project planning and is reviewed by all stakeholders. Quality assurance activities performed by the software engineering team and the SQA group are governed by the plan. The plan identifies evaluations to be performed, audits and reviews to be performed, standards that are applicable to the project, procedures for error reporting and tracking, documents to be produced by the SQA group, and amount of feedback provided to the software project team.

Participates in the development of the project's software process description. The software team selects a process for the work to be performed. The SQA group reviews the process description for compliance with organizational policy, internal software standards, externally imposed standards (e.g., ISO-9001), and other parts of the software project plan.

Reviews software engineering activities to verify compliance with the defined software process. The SQA group identifies, documents, and tracks deviations from the process and verifies that corrections have been made.

Audits designated software work products to verify compliance with those defined as part of the software process. The SQA group reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made; and periodically reports the results of its work to the project manager.

Ensures that deviations in software work and work products are documented and handled according to a documented procedure. Deviations may be encountered in the project plan, process description, applicable standards, or technical work products.

Records any noncompliance and reports to senior management. Noncompliance items are tracked until they are resolved.

In addition to these activities, the SQA group coordinates the control and management of change (Chapter 27) and helps to collect and analyze software metrics.

26.3 SOFTWARE REVIEWS



Reviews are like filters in the software process workflow. Too few, and the flow is "dirty." Too many, and the flow slows to a trickle. Use metrics to determine which reviews work and emphasize them.

Software reviews are a "filter" for the software process. That is, reviews are applied at various points during software engineering and serve to uncover errors and defects that can then be removed. Software reviews "purify" the software engineering activities that we have called analysis, design, and coding. Freedman and Weinberg [FRE90] discuss the need for reviews this way:

Technical work needs reviewing for the same reason that pencils need erasers: *To err is human*. The second reason we need technical reviews is that although people are good at catching some of their own errors, large classes of errors escape the originator more easily than they escape anyone else.

Many different types of reviews can be conducted as part of software engineering. Each has its place. An informal meeting around the coffee machine is a form of review, if technical problems are discussed. A formal presentation of software design to an audience of customers, management, and technical staff is also a form of review. In this book, however, we focus on the *formal technical review*, sometimes called a *walkthrough* or an *inspection*. A formal technical review (FTR) is the most effective filter from a quality assurance standpoint. Conducted by software engineers (and others) for software engineers, the FTR is an effective means for uncovering errors and improving software quality.

INFO



Bugs, Errors, and Defects

The goal of SQA is to remove quality problems in the software. These problems are referred to by various names—"bugs," "faults," "errors," or "defects"—to name a few. Are each of these terms synonymous, or are there subtle differences between them?

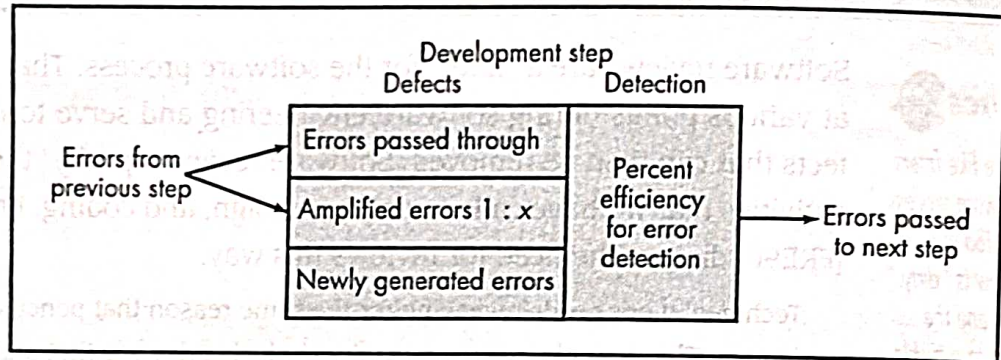
In this book we have made a clear distinction between an *error* (a quality problem found *before* the software is released to end-users) and a *defect* (a quality problem found *only after* the software has been released to end-users²). We make this distinction because errors and defects have very different economic, business, psychological, and human impact. As software engineers, we want to find and correct as many errors as possible before the customer and/or end-user encounter them. We want to avoid defects—because defects (justifiably) make software people look bad.

It is important to note, however, that the temporal distinction made between errors and defects in this book is

not mainstream thinking. The general consensus within the software engineering community is that defects and errors, faults, and bugs are synonymous. That is, the point in time that the problem was encountered has no bearing on the term used to describe the problem. Part of the argument in favor of this view is that it is sometimes difficult to make a clear distinction between pre- and post-release (e.g., consider an incremental process used in agile development [Chapter 4]).

Regardless of how you choose to interpret these terms, recognize that the point in time at which a problem is discovered does matter and that software engineers should try hard—very hard—to find problems before their customers and end-users encounter them. If you have further interest in this issue, a reasonably thorough discussion of the terminology surrounding "bugs" can be found at www.softwaredevelopment.ca/bugs.shtml.

2 If software process improvement is considered, a quality problem that is propagated from one process framework activity (e.g., modeling) to another (e.g., construction) can also be called a "defect" (because the problem should have been found before a work product (e.g., a design model) was "released" to the next activity).

FIGURE 26.2**Defect amplification model****26.3.1 Cost Impact of Software Defects**

The primary objective of an FTR is to find errors before they are passed on to another software engineering activity or released to the end-user.

The primary objective of formal technical reviews is to find errors during the process so that they do not become defects after release of the software. The obvious benefit of formal technical reviews is the early discovery of errors so that they do not propagate to the next step in the software process.

A number of industry studies (by TRW, NEC, Mitre Corp., among others) indicate that design activities introduce between 50 and 65 percent of all errors (and ultimately, all defects) during the software process. However, formal review techniques have been shown to be up to 75 percent effective [JON86] in uncovering design flaws. By detecting and removing a large percentage of these errors, the review process substantially reduces the cost of subsequent activities in the software process.

To illustrate the cost impact of early error detection, we consider a series of relative costs that are based on actual cost data collected for large software projects [IBM81].³ Assume that an error uncovered during design will cost 1.0 monetary unit to correct. Relative to this cost, the same error uncovered just before testing commences will cost 6.5 units; during testing, 15 units; and after release, between 60 and 100 units.

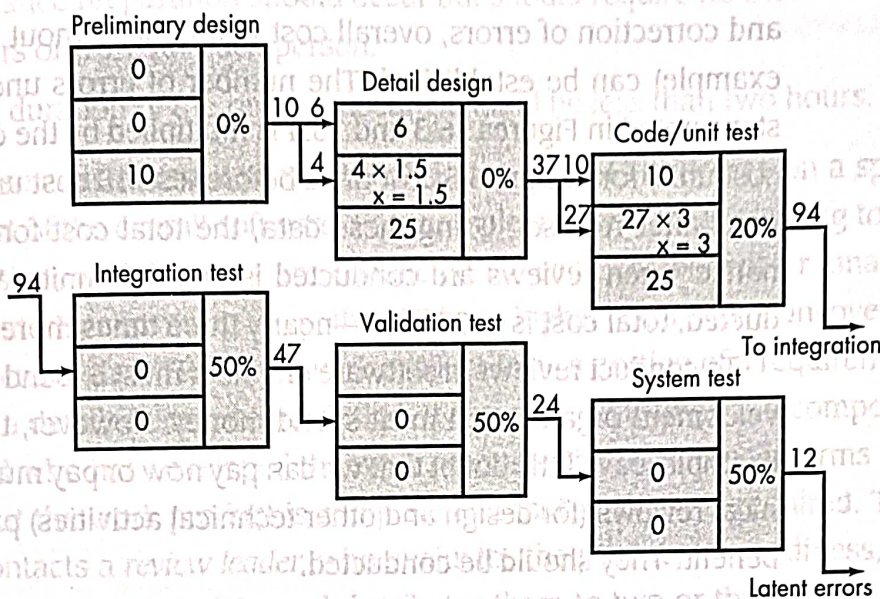
26.3.2 Defect Amplification and Removal

A defect amplification model [IBM81] can be used to illustrate the generation and detection of errors during the preliminary design, detail design, and coding steps of a software engineering process. The model is illustrated schematically in Figure 26.2. A box represents a software development step. During the step, errors may be inadvertently generated. Review may fail to uncover newly generated errors and errors from previous steps, resulting in some number of errors that are passed through. In some cases, errors passed through from previous steps are amplified (amplification factor, x) by current work. The box subdivisions represent each of these characteristics and the percent of efficiency for detecting errors, a function of the thoroughness of the review.

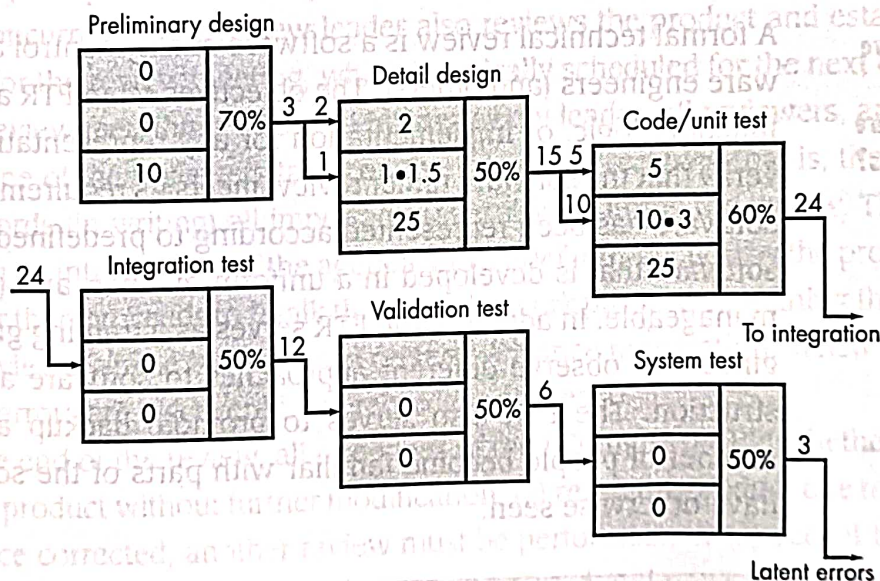
³ Although these data are well over 20 years old, they remain applicable in a modern context.

FIGURE 26.3

Defect amplification—no reviews

**FIGURE 26.4**

Defect amplification—reviews conducted



"Some maladies, as doctors say, at their beginning are easy to cure but difficult to recognize . . . but in the course of time when they have not at first been recognized and treated, become easy to recognize but difficult to cure."

Niccolo Machiavelli

Figure 26.3 illustrates a hypothetical example of defect amplification for a software process in which no reviews are conducted. Referring to the figure, each test step is assumed to uncover and correct 50 percent of all incoming errors without introducing any new errors (an optimistic assumption). Ten preliminary design defects are amplified to 94 errors before testing commences. Twelve latent defects are released to the field. Figure 26.4 considers the same conditions except that design and code reviews are conducted as part of each development step. In this case, 10 initial preliminary design errors are amplified to 24 errors before testing commences. Only

three latent defects exist. Recalling the relative costs associated with the discovery and correction of errors, overall cost (with and without review for our hypothetical example) can be established. The number of errors uncovered during each of the steps noted in Figures 26.3 and 26.4 is multiplied by the cost to remove an error (1.5 cost units for design, 6.5 cost units before test, 15 cost units during test, and 67 cost units after release). Using these data, the total cost for development and maintenance when reviews are conducted is 783 cost units. When no reviews are conducted, total cost is 2177 units—nearly three times more costly.

To conduct reviews, a software engineer must expend time and effort, and the development organization must spend money. However, the results of the preceding example leave little doubt that we can pay now or pay much more later. Formal technical reviews (for design and other technical activities) provide a demonstrable cost benefit. They should be conducted.

26.4 FORMAL TECHNICAL REVIEWS

? When we conduct FTRs, what are our objectives?

A formal technical review is a software quality control activity performed by software engineers (and others). The objectives of an FTR are (1) to uncover errors in function, logic, or implementation for any representation of the software; (2) to verify that the software under review meets its requirements; (3) to ensure that the software has been represented according to predefined standards; (4) to achieve software that is developed in a uniform manner; and (5) to make projects more manageable. In addition, the FTR serves as a training ground, enabling junior engineers to observe different approaches to software analysis, design, and construction. The FTR also serves to promote backup and continuity because a number of people become familiar with parts of the software that they may not have otherwise seen.

"There is no urge so great as for one man to edit another man's work."

Mark Twain

The FTR is actually a class of reviews that includes walkthroughs, inspections, round-robin reviews, and other small group technical assessments of software. Each FTR is conducted as a meeting and will be successful only if it is properly planned, controlled, and attended. In the sections that follow, guidelines similar to those for a walkthrough (e.g., [FRE90], [GIL93]) are presented as a representative formal technical review.

26.4.1 The Review Meeting

Regardless of the FTR format that is chosen, every review meeting should abide by the following constraints:

- Between three and five people (typically) should be involved in the review.

WebRef

The NASA SATC Formal Inspection Guidebook can be downloaded at satc.gsfc.nasa.gov/ti/tipage.html.

- Advance preparation should occur but should require no more than two hours of work for each person.
- The duration of the review meeting should be less than two hours.

Given these constraints, it should be obvious that an FTR focuses on a specific (and small) part of the overall software. For example, rather than attempting to review an entire design, walkthroughs are conducted for each component or small group of components. By narrowing focus, the FTR has a higher likelihood of uncovering errors.

The focus of the FTR is on a work product (e.g., a portion of a requirements specification, a detailed component design, a source code listing for a component). The individual who has developed the work product—the *producer*—informs the project leader that the work product is complete and that a review is required. The project leader contacts a *review leader*, who evaluates the product for readiness, generates copies of product materials, and distributes them to two or three *reviewers* for advance preparation. Each reviewer is expected to spend between one and two hours reviewing the product, making notes, and otherwise becoming familiar with the work. Concurrently, the review leader also reviews the product and establishes an agenda for the review meeting, which is typically scheduled for the next day.

The review meeting is attended by the review leader, all reviewers, and the producer. One of the reviewers takes on the role of the *recorder*; that is, the individual who records (in writing) all important issues raised during the review. The FTR begins with an introduction of the agenda and a brief introduction by the producer. The producer then proceeds to “walk through” the work product, explaining the material, while reviewers raise issues based on their advance preparation. When valid problems or errors are discovered, the recorder notes each.

At the end of the review, all attendees of the FTR must decide whether to (1) accept the product without further modification, (2) reject the product due to severe errors (once corrected, another review must be performed), or (3) accept the product provisionally (minor errors have been encountered and must be corrected, but no additional review will be required). The decision made, all FTR attendees complete a sign-off, indicating their participation in the review and their concurrence with the review team’s findings.

26.4.2 Review Reporting and Record Keeping

During the FTR, a reviewer (the recorder) actively records all issues that have been raised. These are summarized at the end of the review meeting and a *review issues list* is produced. In addition, a *formal technical review summary report* is completed. A review summary report answers three questions:

1. What was reviewed?
2. Who reviewed it?
3. What were the findings and conclusions?

KEY POINT

An FTR focuses on a relatively small portion of a work product.

ADVICE

In some situations, it’s a good idea to have someone other than the producer walk through the product undergoing review. This leads to a literal interpretation of the work product and better error recognition.

The review summary report is a single page form (with possible attachments). It becomes part of the project historical record and may be distributed to the project leader and other interested parties.

The review issues list serves two purposes: (1) to identify problem areas within the product and (2) to serve as an action item checklist that guides the producer as corrections are made. An issues list is normally attached to the summary report.

It is important to establish a follow-up procedure to ensure that items on the issues list have been properly corrected. Unless this is done, it is possible that issues raised can "fall between the cracks." One approach is to assign the responsibility for follow-up to the review leader.

"A meeting is too often an event in which minutes are taken and hours are wasted."

Author unknown

26.4.3 Review Guidelines

Guidelines for conducting formal technical reviews must be established in advance, distributed to all reviewers, agreed upon, and then followed. A review that is uncontrolled can often be worse than no review at all. The following represents a minimum set of guidelines for formal technical reviews:



Don't point out errors harshly. One way to be gentle is to ask a question that enables the producer to discover the error.

1. *Review the product, not the producer.* An FTR involves people and egos. Conducted properly, the FTR should leave all participants with a warm feeling of accomplishment. Conducted improperly, the FTR can take on the aura of an inquisition. Errors should be pointed out gently; the tone of the meeting should be loose and constructive; the intent should not be to embarrass or belittle.
2. *Set an agenda and maintain it.* One of the key maladies of meetings of all types is drift. An FTR must be kept on track and on schedule. The review leader is chartered with the responsibility for maintaining the meeting schedule and should not be afraid to nudge people when drift sets in.
3. *Limit debate and rebuttal.* When an issue is raised by a reviewer, there may not be universal agreement on its impact. Rather than spending time debating the question, the issue should be recorded for further discussion off-line.
4. *Enunciate problem areas, but don't attempt to solve every problem noted.* A review is not a problem-solving session. Problem solving should be postponed until after the review meeting.
5. *Take written notes.* It is sometimes a good idea for the recorder to make notes on a wall board, so that wording and priorities can be assessed by other reviewers as information is recorded.
6. *Limit the number of participants and insist upon advance preparation.* Two heads are better than one, but 14 are not necessarily better than 4. Keep the number of people involved to the necessary minimum. However, all review

team members must prepare in advance. Written comments should be solicited by the review leader (providing an indication that the reviewer has reviewed the material).

7. *Develop a checklist for each product that is likely to be reviewed.* A checklist helps the review leader to structure the FTR meeting and helps each reviewer to focus on important issues.
8. *Allocate resources and schedule time for FTRs.* For reviews to be effective, they should be scheduled as a task during the software process. In addition, time should be scheduled for the inevitable modifications that will occur as the result of an FTR.
9. *Conduct meaningful training for all reviewers.* To be effective all review participants should receive some formal training. The training should stress both process-related issues and the human psychological side of reviews.
10. *Review your early reviews.* Debriefing can be beneficial in uncovering problems with the review process itself. The very first product to be reviewed should be the review guidelines themselves.

"It is one of the most beautiful compensations of life, that no man can sincerely try to help another without helping himself."

Ralph Waldo Emerson

Because many variables (e.g., number of participants, type of work products, timing and length, specific review approach) have an impact on a successful review, a software organization should experiment to determine what approach works best in a local context. Porter and his colleagues [POR95] provide excellent guidance for this type of experimentation.

26.4.4 Sample-Driven Reviews

In an ideal setting, every software engineering work product would undergo a formal technical review. In the real world of software projects, resources are limited and time is short. As a consequence, reviews are often skipped, even though their value as a quality control mechanism is recognized. Thelin and his colleagues [THE01] address this issue when they state:

Inspections [FTRs] are only viewed efficient if many faults are found during the fault searching part. If many faults are found in the artifacts [work products], the inspections are necessary. If, on the other hand, only few faults are found, the inspection has been a waste of time for several people involved in the inspections⁴. Moreover, software projects which are late often decrease the time for inspection activities, which leads to a lack of

⁴ Of course, it can be argued that by conducting reviews we encourage producers to focus on quality, even if no errors are found.

quality. A solution would be to prioritize the resources for the inspection activities and thereby concentrate the available resources on the artifacts that are most fault-prone.



ADVICE
Reviews take time, but it's time well spent. However, if time is short and you have no other option, do not dispense with reviews. Rather, use sample-driven reviews.

Thelin and his colleagues suggest a sample-driven review process in which samples of all software engineering work products are inspected to determine which work products are most error prone. Full FTR resources are then focused only on those work products that are likely (based on data collected during sampling) to be error-prone.

To be effective, the sample driven review process must attempt to quantify those work products that are primary targets for full FTRs. To accomplish this, the following steps are suggested [THE01]:

1. Inspect a fraction a_i of each software work product, i . Record the number of faults, f_i found within a_i .
2. Develop a gross estimate of the number of faults within work product i by multiplying f_i by $1/a_i$.
3. Sort the work products in descending order according to the gross estimate of the number of faults in each.
4. Focus available review resources on those work products that have the highest estimated number of faults.

The fraction of the work product that is sampled must (1) be representative of the work product as a whole and (2) large enough to be meaningful to the reviewer(s) who does the sampling. As a_i increases, the likelihood that the sample is a valid representation of the work product also increases. However, the resources required to do sampling also increase. A software engineering team must establish the best value for a_i for the types of work products produced.⁵

SAFEHOME



SQA Issues

The scene: Doug Miller's office as the SafeHome software project begins.

The players: Doug Miller (manager of the SafeHome software engineering team) and other members of the software engineering team.

The conversation:

Doug: I know we didn't spend time developing an SQA plan for this project, but we're already into it and we have to consider quality . . . right?

Jamie: Sure. We've already decided that as we develop the requirements model [Chapters 7 and 8], Ed has committed to develop a V&V procedure for each requirement.

Doug: That's really good, but we're not going to wait until testing to evaluate quality, are we?

Vinod: No! Of course not. We've got reviews scheduled into the project plan for this software increment. We'll begin quality control with the reviews.

⁵ Thelin and his colleagues have conducted a detailed simulation that can assist in making this determination. See [THE01] for details.

Jamie: I'm a bit concerned that we won't have enough time to conduct all the reviews. In fact, I know we won't.

Doug: Hmmm. So what do you propose?

Jamie: I say we select those elements of the analysis and design model that are most critical to *SafeHome* and review them.

Vinod: But what if we miss something in a part of the model we don't review?

Shakira: I read something about a sampling technique [Section 26.4.4] that might help us target candidates for review. (Shakira explains the approach.)

Jamie: Maybe . . . but I'm not sure we even have time to sample every element of the models.

Vinod: What do you want us to do, Doug?

Doug: Let's steal something from Extreme Programming [Chapter 4]. We'll develop the elements of each model in pairs—two people—and conduct an informal review of each as we go. We'll then target "critical" elements for a more formal team review, but keep those reviews to a minimum. That way, everything gets looked at by more than one set of eyes, but we still maintain our delivery dates.

Jamie: That means we're going to have to revise the schedule.

Doug: So be it. Quality trumps schedule on this project.

26.5 FORMAL APPROACHES TO SQA

Over the past two decades, a small, but vocal, segment of the software engineering community has argued that a more formal approach to software quality assurance is required. It can be argued that a computer program is a mathematical object [SOM01]. A rigorous syntax and semantics can be defined for every programming language, and a rigorous approach to the specification of software requirements (Chapter 28) is available. If the requirements model (specification) and the programming language can be represented in a rigorous manner, it should be possible to apply mathematic proof of correctness to demonstrate that a program conforms exactly to its specifications.

Attempts to prove programs correct (Chapters 28 and 29) are not new. Dijkstra [DIJ76] and Linger, Mills, and Witt [LIN79], among others, advocated proofs of program correctness and tied these to the use of structured programming concepts (Chapter 11).

26.6 STATISTICAL SOFTWARE QUALITY ASSURANCE

Statistical quality assurance reflects a growing trend throughout industry to become more quantitative about quality. For software, statistical quality assurance implies the following steps:

1. Information about software defects is collected and categorized.
2. An attempt is made to trace each defect to its underlying cause (e.g., non-conformance to specifications, design error, violation of standards, poor communication with the customer).

? What steps are required to perform statistical SQA?

3. Using the Pareto principle (80 percent of the defects can be traced to 20 percent of all possible causes), isolate the 20 percent (the "vital few").
4. Once the vital few causes have been identified, move to correct the problems that have caused the defects.

This relatively simple concept represents an important step towards the creation of an adaptive software process in which changes are made to improve those elements of the process that introduce error.

"20 percent of the code has 80 percent of the errors. Find them, fix them!"

Lowell Arthur

26.6.1 A Generic Example

To illustrate the use of statistical methods for software engineering work, assume that a software engineering organization collects information on defects for a period of one year. Some of the defects are uncovered as software is being developed. Others are encountered after the software has been released to its end-users. Although hundreds of different defects are uncovered, all can be tracked to one (or more) of the following causes:

- Incomplete or erroneous specifications (IES).
- Misinterpretation of customer communication (MCC).
- Intentional deviation from specifications (IDS).
- Violation of programming standards (VPS).
- Error in data representation (EDR).
- Inconsistent component interface (ICI).
- Error in design logic (EDL).
- Incomplete or erroneous testing (IET).
- Inaccurate or incomplete documentation (IID).
- Error in programming language translation of design (PLT).
- Ambiguous or inconsistent human/computer interface (HCI).
- Miscellaneous (MIS).

To apply statistical SQA, the table in Figure 26.5 is built. The table indicates that IES, MCC, and EDR are the vital few causes that account for 53 percent of all errors. It should be noted, however, that IES, EDR, PLT, and EDL would be selected as the vital few causes if only serious errors are considered. Once the vital few causes are determined, the software engineering organization can begin corrective action. For example, to correct MCC, the software developer might implement facilitated requirements gathering techniques (Chapter 7) to improve the quality of customer communication and specifications. To improve EDR, the developer

FIGURE 26.5Data collection
for statistical
SQA

Error	Total		Serious		Moderate		Minor	
	No.	%	No.	%	No.	%	No.	%
IES	205	22%	34	27%	68	18%	103	24%
MCC	156	17%	12	9%	68	18%	76	17%
IDS	48	5%	1	1%	24	6%	23	5%
VPS	25	3%	0	0%	15	4%	10	2%
EDR	130	14%	26	20%	68	18%	36	8%
ICI	58	6%	9	7%	18	5%	31	7%
EDL	45	5%	14	11%	12	3%	19	4%
IET	95	10%	12	9%	35	9%	48	11%
IID	36	4%	2	2%	20	5%	14	3%
PLT	60	6%	15	12%	19	5%	26	6%
HCI	28	3%	3	2%	17	4%	8	2%
MIS	56	6%	0	0%	15	4%	41	9%
Totals	942	100%	128	100%	379	100%	435	100%

might acquire tools for data modeling and perform more stringent data design reviews.

It is important to note that corrective action focuses primarily on the vital few. As the vital few causes are corrected, new candidates pop to the top of the stack.

Statistical quality assurance techniques for software have been shown to provide substantial quality improvement [ART97]. In some cases, software organizations have achieved a 50 percent reduction per year in defects after applying these techniques.

The application of the statistical SQA and the Pareto principle can be summarized in a single sentence: *Spend your time focusing on things that really matter, but first be sure that you understand what really matters!*

A comprehensive discussion of statistical SQA is beyond the scope of this book. Interested readers should see [GOH02], [SCH98], or [KAN95].

26.6.2 Six Sigma for Software Engineering

Six Sigma is the most widely used strategy for statistical quality assurance in industry today. Originally popularized by Motorola in the 1980s, the Six Sigma strategy "is a rigorous and disciplined methodology that uses data and statistical analysis to measure and improve a company's operational performance by identifying and eliminating 'defects' in manufacturing and service-related processes." [ISI03]. The term "six sigma" is derived from six standard deviations—3.4 instances (defects) per million occurrences—implying an extremely high quality standard. The Six Sigma methodology defines three core steps:

- Define customer requirements, deliverables, and project goals via well-defined methods of customer communication.

? What are the core steps of the six sigma methodology?

- *Measure* the existing process and its output to determine current quality performance (collect defect metrics).
- *Analyze* defect metrics and determine the vital few causes.

If an existing software process is in place, but improvement is required, Six Sigma suggests two additional steps:

- *Improve* the process by eliminating the root causes of defects.
- *Control* the process to ensure that future work does not reintroduce the causes of defects.

These core and additional steps are sometimes referred to as the DMAIC (define, measure, analyze, improve, and control) method.

If an organization is developing a software process (rather than improving an existing process), the core steps are augmented as follows:

- *Design* the process to (1) avoid the root causes of defects and (2) to meet customer requirements
- *Verify* that the process model will, in fact, avoid defects and meet customer requirements.

This variation is sometimes called the DMADV (define, measure, analyze, design, and verify) method.

A comprehensive discussion of Six Sigma is best left to resources dedicated to the subject. The interested reader should see [ISI03], [SNE03], and [PAN00].

26.7 SOFTWARE RELIABILITY

Software reliability, unlike many other quality factors, can be measured directly and estimated using historical and developmental data. *Software reliability* is defined in statistical terms as "the probability of failure-free operation of a computer program in a specified environment for a specified time" [MUS87]. To illustrate, program X is estimated to have a reliability of 0.96 over eight elapsed processing hours. In other words, if program X were to be executed 100 times and require a total of eight hours of elapsed processing time (execution time), it is likely to operate correctly (without failure) 96 times.

"The unavoidable price of reliability is simplicity."

C.A.R. Hoare

Whenever software reliability is discussed, a pivotal question arises: What is meant by the term *failure*? In the context of any discussion of software quality and reliability, failure is nonconformance to software requirements. Yet, even within this definition, there are gradations. Failures can be only annoying or catastrophic. One failure can

be corrected within seconds while another requires weeks or even months to correct. Complicating the issue even further, the correction of one failure may in fact result in the introduction of other errors that ultimately result in other failures.

26.7.1 Measures of Reliability and Availability

Early work in software reliability attempted to extrapolate the mathematics of hardware reliability theory (e.g., [ALV64]) to the prediction of software reliability. Most hardware-related reliability models are predicated on failure due to wear rather than failure due to design defects. In hardware, failures due to physical wear (e.g., the effects of temperature, corrosion, shock) are more likely than a design-related failure. Unfortunately, the opposite is true for software. In fact, all software failures can be traced to design or implementation problems; wear (Chapter 1) does not enter into the picture.

There has been debate over the relationship between key concepts in hardware reliability and their applicability to software (e.g., [LIT89], [ROO90]). Although an irrefutable link has yet to be established, it is worthwhile to consider a few simple concepts that apply to both system elements.

If we consider a computer-based system, a simple measure of reliability is *mean-time-between-failure* (MTBF), where

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

The acronyms MTTF and MTTR are *mean-time-to-failure* and *mean-time-to-repair*,⁶ respectively.

Many researchers argue that MTBF is a far more useful measure than defects/KLOC or defects/FP. Stated simply, an end-user is concerned with failures, not with the total error count. Because each defect contained within a program does not have the same failure rate, the total defect count provides little indication of the reliability of a system.

In addition to a reliability measure, we must develop a measure of availability. *Software availability* is the probability that a program is operating according to requirements at a given point in time and is defined as

$$\text{Availability} = [\text{MTTF}/(\text{MTTF} + \text{MTTR})] \times 100\%$$

The MTBF reliability measure is equally sensitive to MTTF and MTTR. The availability measure is somewhat more sensitive to MTTR, an indirect measure of the maintainability of software.

26.7.2 Software Safety

Software safety [LEV86] is a software quality assurance activity that focuses on the identification and assessment of potential hazards that may affect software negatively and cause an entire system to fail. If hazards can be identified early in the software

6 Although debugging (and related corrections) may be required as a consequence of failure, in many cases the software will work properly after a restart with no other change.

KEY POINT

Software reliability problems can almost always be traced to defects in design or implementation.

KEY POINT

It is important to note that MTBF and related measures are based on CPU time, not wall clock time.

ADVICE

Some aspects of availability (not discussed here) have nothing to do with failure. For example, schedule downtime (for support functions) causes the software to be unavailable.

process, software design features can be specified that will either eliminate or control potential hazards.

**"I cannot imagine any condition which would cause this ship to founder. Modern shipbuilding has gone beyond that."
E. I. Smith, captain of the Titanic**

A modeling and analysis process is conducted as part of software safety. Initially, hazards are identified and categorized by criticality and risk. For example, some of the hazards associated with a computer-based cruise control for an automobile might be:

- Causes uncontrolled acceleration that cannot be stopped.
- Does not respond to depression of brake pedal (by turning off).
- Does not engage when switch is activated.
- Slowly loses or gains speed.

Once these system-level hazards are identified, analysis techniques are used to assign severity and probability of occurrence.⁷ To be effective, software must be analyzed in the context of the entire system. For example, a subtle user input error (people are system components) may be magnified by a software fault to produce control data that improperly positions a mechanical device. If a set of external environmental conditions are met (and only if they are met), the improper position of the mechanical device will cause a disastrous failure. Analysis techniques such as fault tree analysis [VES81], real-time logic [JAN86], or Petri net models [LEV87] can be used to predict the chain of events that can cause hazards and the probability that each of the events will occur to create the chain.

Once hazards are identified and analyzed, safety-related requirements can be specified for the software. That is, the specification can contain a list of undesirable events and the desired system responses to these events. The role of software in managing undesirable events is then indicated.

Although software reliability and software safety are closely related to one another, it is important to understand the subtle difference between them. Software reliability uses statistical analysis to determine the likelihood that a software failure will occur. However, the occurrence of a failure does not necessarily result in a hazard or mishap. Software safety examines the ways in which failures result in conditions that can lead to a mishap. That is, failures are not considered in a vacuum, but are evaluated in the context of an entire computer-based system and its environment. Those readers with further interest should refer to Leveson's [LEV95] book on the subject.

WebRef

A worthwhile collection of papers on software safety can be found at www.safeware-eng.com/.

⁷ This approach is similar to the risk analysis methods described in Chapter 25. The primary difference is the emphasis on technology issues rather than project related topics.

26.8 THE ISO 9000 QUALITY STANDARDS⁸

A *quality assurance system* may be defined as the organizational structure, responsibilities, procedures, processes, and resources for implementing quality management [ANS87]. Quality assurance systems are created to help organizations ensure their products and services satisfy customer expectations by meeting their specifications. ISO 9000 describes a quality assurance system in generic terms that can be applied to any business regardless of the products or services offered.

To become registered to one of the quality assurance system models contained in ISO 9000, a company's quality system and operations are scrutinized by third-party auditors for compliance to the standard and for effective operation. Upon successful registration, a company is issued a certificate from a registration body represented by the auditors. Semiannual surveillance audits ensure continued compliance to the standard.

ISO 9001:2000 is the quality assurance standard that applies to software engineering. The standard contains 20 requirements that must be present for an effective quality assurance system. Because the ISO 9001:2000 standard is applicable to all engineering disciplines, a special set of ISO guidelines (ISO 9000-3) have been developed to help interpret the standard for use in the software process.

The requirements delineated by ISO 9001:2000 address topics such as management responsibility, quality system, contract review, design control, document and data control, product identification and traceability, process control, inspection and testing, corrective and preventive action, control of quality records, internal quality audits, training, servicing, and statistical techniques. For a software organization to become registered to ISO 9001:2000, it must establish policies and procedures to address each of the requirements just noted (and others) and then be able to demonstrate that these policies and procedures are being followed. For further information on ISO 9001, the interested reader should see [HOY02], [GAA01], or [CIA01].

KEY POINT

ISO 9000 describes what must be done to be compliant, but it does not describe how it must be done.

WebRef

Extensive links to ISO 9000/9001 resources can be found at www.tamara.ab.ca/info.htm.

INFO



The ISO 9001:2000 Standard

The following outline defines the basic elements of the ISO 9001:2000 standard. Comprehensive information on the standard can be obtained from the International Organization for Standardization (www.iso.ch) and other Internet sources (e.g., www.praxiom.com).

Establish the elements of a quality management system.
Develop, implement, and improve the system.

- Define a policy that emphasizes the importance of the system.
- Document the quality system.
- Describe the process.
- Produce an operational manual.
- Develop methods for controlling (updating) documents.
- Establish methods for recordkeeping.
- Support quality control and assurance.

⁸ This section, written by Michael Stovsky, has been adapted from "Fundamentals of ISO 9000," a workbook developed for *Essential Software Engineering*, a video curriculum developed by R. S. Pressman & Associates, Inc. Reprinted with permission.

- | | |
|--|---|
| Promote the importance of quality among all stakeholders. | Establish control mechanisms. |
| Focus on customer satisfaction. | For planning. |
| Define a quality plan that addresses objectives, responsibilities, and authority. | For customer requirements. |
| Define communication mechanisms among stakeholders. | For technical activities (e.g., analysis, design, testing). |
| Establish review mechanisms for the quality management system. | For project monitoring and management. |
| Identify review methods and feedback mechanisms. | Define methods for remediation. |
| Define follow-up procedures. | Assess quality data and metrics. |
| Identify quality resources including personnel, training, infrastructure elements. | Define approach for continuous process and quality improvement. |

26.9 THE SQA PLAN

The *SQA Plan* provides a road map for instituting software quality assurance. Developed by the SQA group (or the software team if a SQA group does not exist), the plan serves as a template for SQA activities that are instituted for each software project.

A standard for SQA plans has been published by the IEEE [IEE94]. The standard recommends a structure that identifies (1) the purpose and scope of the plan; (2) a description of all software engineering work products (e.g., models, documents, source code) that fall within the purview of SQA; (3) all applicable standards and practices that are applied during the software process; (4) SQA actions and tasks (including reviews and audits) and their placement throughout the software process; (5) the tools and methods that support SQA actions and tasks; (6) software configuration management procedures (Chapter 27) for managing change; (7) methods for assembling, safeguarding, and maintaining all SQA-related records; and (8) organizational roles and responsibilities relative to product quality.

SOFTWARE TOOLS



Software Quality Management

Objective: The objective of SQA tools is to assist a project team in assessing and improving the quality of software work product.

Mechanics: Tools mechanics vary. In general, the intent is to assess the quality of a specific work product. Note: a

wide array of software testing tools (see Chapters 13 and 14) are often included within the SQA tools category.

Representative Tools⁹

ARM, developed by NASA

(satc.gsfc.nasa.gov/tools/index.html), provides

⁹ Tools noted here do not represent an endorsement, but rather a sampling of tools in this category. In most cases, tool names are trademarked by their respective developers.

measures that can be used to assess the quality of a software requirements document.

QPR ProcessGuide and Scorecard, developed by QPR Software (www.qpronline.com), provides support for Six Sigma and other quality management approaches.

Quality Tools Cookbook, developed by Systma and Manley (www.sytma.com/tqmtools/tqmtoolmenu.html), provides useful descriptions of classic quality management tools such as control charts, scatter diagrams, affinity diagrams, and matrix diagrams.

Quality Tools and Templates, developed by iSixSigma (<http://www.isixsigma.com/tt/>), describe a wide array of useful tools and methods for quality management.

TQM Tools, developed by Bain & Company (www.bain.com), provide useful descriptions of a variety of management tools used for TQM and related quality management methods.

26.10 SUMMARY

Software quality management is an umbrella activity—incorporating both quality control and quality assurance—that is applied at each step in the software process. SQA encompasses procedures for the effective application of methods and tools, formal technical reviews, testing strategies and techniques, procedures for change control, procedures for assuring compliance to standards, and measurement and reporting mechanisms.

SQA is complicated by the complex nature of software quality—an attribute of computer programs that is defined as “conformance to explicitly and implicitly specified requirements.” But when considered more generally, software quality encompasses many different product and process factors and related metrics.

Software reviews are one of the most important quality control activities. Reviews serve as filters throughout all software engineering activities, removing errors while they are relatively inexpensive to find and correct. The formal technical review is a stylized meeting that has been shown to be extremely effective in uncovering errors.

To properly conduct software quality assurance, data about the software engineering process should be collected, evaluated, and disseminated. Statistical SQA helps to improve the quality of the product and the software process itself. Software reliability models extend measurements, enabling collected defect data to be extrapolated into projected failure rates and reliability predictions.

In summary, we recall the words of Dunn and Ullman [DUN82]: “Software quality assurance is the mapping of the managerial precepts and design disciplines of quality assurance onto the applicable managerial and technological space of software engineering.” The ability to ensure quality is the measure of a mature engineering discipline. When the mapping is successfully accomplished, mature software engineering is the result.

be downloaded from <http://www.cas.mcmaster.ca/wise/wise01/ThelinPetersson-Wohlin.pdf>.

[VES81] Veseley, W. E., et al., *Fault Tree Handbook*, U.S. Nuclear Regulatory Commission, NUREG-0492, January 1981.

PROBLEMS AND POINTS TO PONDER

26.1. Some people argue that an FTR should assess programming style as well as correctness. Is this a good idea? Why?

26.2. Is it possible to assess the quality of software if the customer keeps changing what it is supposed to do?

26.3. Acquire a copy of ISO 9001:2000 and ISO 9000-3. Prepare a presentation that discusses three ISO 9001 requirements and how they apply in a software context.

26.4. Early in this chapter we noted that "variation control is the heart of quality control." Since every program that is created is different from every other program, what are the variations that we look for and how do we control them?

26.5. Can a program be correct and still not exhibit good quality? Explain.

26.6. Why is there often tension between a software engineering group and an independent software quality assurance group? Is this healthy?

26.7. You have been given the responsibility for improving the quality of software across your organization. What is the first thing that you should do? What's next?

26.8. The MTBF concept for software is open to criticism. Can you think of a few reasons why?

26.9. A formal technical review is effective only if everyone has prepared in advance. How do you recognize a review participant who has not prepared? What do you do if you're the review leader?

26.10. Quality and reliability are related concepts but are fundamentally different in a number of ways. Discuss them.

26.11. Consider two safety critical systems that are controlled by computers. List at least three hazards for each that can be directly linked to software failures.

26.12. Research the literature on software reliability, and write a paper that describes one software reliability model. Be sure to provide an example.

26.13. Review the table presented in Figure 26.5 and select four vital few causes of serious and moderate errors. Suggest corrective actions using information presented in other chapters.

26.14. Besides counting errors and defects, are there other countable characteristics of software that imply quality? What are they, and can they be measured directly?

26.15. Can a program be correct and still not be reliable? Explain.

FURTHER READINGS AND INFORMATION SOURCES

Books by Moriguchi (*Software Excellence: A Total Quality Management Guide*, Productivity Press, 1997) and Horch (*Practical Guide to Software Quality Management*, Artech Publishing, 1996) are excellent management-level presentations on the benefits of formal quality assurance programs for computer software. Books by Deming [DEM86], Juran (*Juran on Quality by Design*, Free Press, 1992), and Crosby [CRO79] and *Quality Is Still Free*, McGraw-Hill, 1995) do not focus on software, but are must reading for senior managers with software development responsibility. Gluckman and Roome (*Everyday Heroes of the Quality Movement*, Dorset House, 1993) humanizes quality issues by telling the story of the players in the quality process. Kan (*Metrics and Models in Software Quality Engineering*, Addison-Wesley, 1995) presents a quantitative view of software quality.